Infosys®

POWERED BY INTELLECT
DRIVEN BY VALUES

RDBMS – Day7

Time Stamping and Database Recovery

Education and Research

A computer system may fail due to disk crash, power failure etc. A recovery scheme is an integral part of every DBMS . This minimizes the time duration required for the database to become usable after a crash.

Time stamping

- Mechanism for serialization of a set of transactions in the chronological order of start time of these transactions.
- Each data item is associated with two values:
  - Wx: the largest timestamp value of any transaction that was allowed to write a value of X
  - Rx : the largest timestamp value of any transaction that was allowed to read the current value of X
- Timestamp could be based on system clock

Copyright © 2004, Infosys Technologies Ltd

The time stamp protocol allows conflicting trnasactions to proceed based on time stamp ordering. The idea is like this:

Let us say transaction Ti issues READ(X)

Case I:

If TS(Ti) < Wx(X), that means transaction Ti is wanting to read the value of X, but before it could do so, a later transaction has modified the value of X. So, what was the value of X when Ti started would be different from what the value of X is now, when ti actually gets a chance to read. So, this read request is rejected.

If TS(Ti) >= Wx(x) then read operation is executed and Rx(x) is updated to Max(Rx(x), Ts(Ti)

Continued in the notes page of the next slide……

Two principles of Timestamping

- A transaction can read only rows or columns that have been updated by an older transaction else this transaction is rolled back.
- A transaction can update only rows or columns that have been read and updated by an older transaction else this transaction is rolled back.

Concerns in time-stamping

- Why is there a high rate of transaction roll-back?
- Why is there a high level of concurrency?
- Why are there no deadlocks?

Copyright © 2004, Infosys Technologies Ltd

ER/CORP/CRS/DB07/003
Version No: 2.0

To explain this with an example, let us say a data item X has the following status

X WX=3.40 p.m Rx= 3.20 p.m

Now transaction Ti starts at 3.35 p.m who's action to be performed is Read(x). Ti wanted to read x at 3.35. at that time the value could have been 20 (assume) so Ti actually wanted this value 20. But, Ti could not get a chance at that time. In the meantime, another transaction Tj has updated the value of X at 3.40 p.m . When Ti is getting the chance, it is already 3.42 p.m .. If Ti reads the value of x, it is going to be the new valueas updated by tj and not the one ti actually wanted to read. So, there is no point in performing this read operation now. Ti is rejected and has to try again.

Let us say Ti issues Write(Q)

Case 1: if(Ts(Ti) < Rx(x) it means ti is trying to update x whereas some other trans has already read the value of x. so the updation done by ti is not required. So ti is rejected

Case 2: if Ts(Ti) < Wx(x) then it means before Ti could update x, some other transaction has updated x with a latest value. So ti is rejected

Case 3 : else ti is executed.

Difference between Locking and Timestamping

LOCKING	TIMESTAMPING
1. Guarantees some serializability	1. Guarantees serializability based on birth time of successfully completed transactions
2. Possibility of deadlocks	2. No deadlocks
3. Lower level of concurrency	3. Higher level of concurrency
4. Lower rate of transaction rollbacks	4. Higher rate of transaction rollbacks (Why?)

Copyright © 2004, Infosys Technologies Ltd

Security of Data

- USERID and PASSWORD to restrict the users from acquiring an un-authorized access
- Grant and Revoke statements (Data Control Language) to provide restricted access control to resources like Tables
- Database views to restrict access to sensitive data
- Encryption of data to avoid un-authorized access

Failure types

- Transaction failure
- System crash
- Disk failure

Copyright © 2004, Infosys Technologies Ltd

Transaction failure:

Logical error: overflow, resource limit exceeded, wrong input etc.

System error: deadlock

System crash: hardware errors, bug in database software etc which stop the transaction from proceeding. In this case, the data in the database (on the disk) is intact

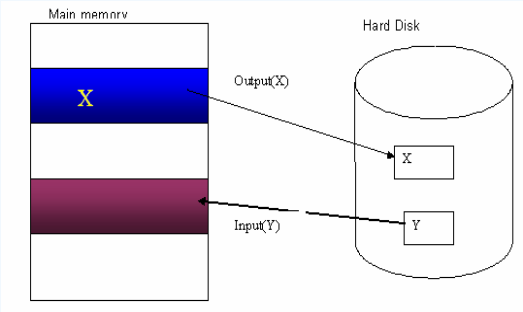
Disk failure: data is lost from the disk blocks due to head crash etc.

Two things should be taken care in recovery algorithms:

1. Actions to be taken during normal execution, to backup enough information for a future recovery in case of a failure
2. Actions taken after a failure to restore the database contents to a consistent state

Storage Types

- **Volatile**
  - Is fast
  - E.g. Main memory, Cache
- **Non-Volatile**
  - Survives system crashes
  - E.g. disk, magnetic tapes
- **Stable**
  - Theoretically never fails
  - E.g. RAID, remote back up systems etc.



Copyright © 2004, Infosys Technologies Ltd

•The database resides on hard disk
•It is partitioned into fixed size units called blocks
•A single block may contain many data items
•Transactions read from and write to the disk in terms of blocks
•The blocks on the disk are called **disk blocks** and those on the primary memory (ie those copied from the disk into primary memory) are called **buffer blocks.** The area of memory where such buffers reside is called the **'disk buffer'/DBMS Cache**

•To read /write a block from /to the disk, the transaction has to do the following operations:

Input(B) : transfers contents of the disk block B into primary memory
Output(B): transfers the buffer block B from the primary memory to disk, overwriting its contents

When a transaction is initiated, system creates a private work area for the transaction in which copies of all data items accessed and updated by Ti are stored. This area is removed when the transaction commits or aborts

Let us say, a data item X kept in the work area of transaction Ti is represented as xi

Transaction Ti interacts with the database system by transferring data to and from its work area to the disk buffer.

Read (X) : Assigns the value of data item X from the disk buffer to the local variable xi in the work area of the transaction Ti
Write (X) : Assigns the value of xi to X in the buffer Bx in the disk buffer.

The updation done by the transaction on the local variable xi is reflected on the original variable X when the transaction says Write(X)

The Transaction Log

- Most important structure used to recover from database failures
- Contains the before image and after image of the data item modified
- Contains log records for each update activity
- Each log record contains:
  - Transaction identifier
  - Data item identifier
  - Old value
  - New value <Ti, Xi, V1, v2>
- Start and commit are indicated with special records

Copyright © 2004, Infosys Technologies Ltd

As soon as a transaction performs a write, it should create the log record and store the log record on stable storage.

Techniques to maintain log files

- Deferred Update
- Immediate Update

Deferred Update Scheme (1of 2)

- Database modifications are written in the log,
- The WRITE operation is deferred until a transaction **Partially Commits**
- A transaction is said to be partially committed when it completes its final action
- Before the actual updates on the database begins, it must be ensured that all log records are written to the stable storage
- Once all the updates are done on the database the transaction enters the 'committed' state.

Copyright © 2004, Infosys Technologies Ltd

When the transaction partially COMMITs, the information associated with it is used to update the database .If a transaction aborts or system crashes before a transaction completes, the information on the log is IGNORED

Consider a fund transfer transaction to transfer rs 100 from account A to account B. The initial value in account A is 1000 and that in B is 2000. This will be followed by a deposit transaction which deposits rs 100 to another account C. initial value of c is rs 1000.

A= 1000
B= 2000
C= 1000

Now, the two transactions would perform steps as below:

T0: read(A);	T1: Read(C);
A:=A-100;	C:=C+100;
write(A);	write(C)
read(B);	
B:= B+100;	
write(B)	

Since in this scheme, the update to the actual database happens only after the transaction commits, the old value is not required to be stored. Just the new value is enough.

If the two transactions execute successfully, then the snapshot of the log and the database would be like:

Log	Database
<T0 start>	
<T0 A,900>	
<T0 B, 2100>	
<To Commit>	
	A= 900
	B= 2100
<T1 start>	
<T1 C, 1100>	
<T1 Commit>	
	C= 1100

Note that the value of the data item is changed in the database only after the <Ti commit> record is written to the log .

Deferred Update Scheme (2 of 2)

- The recovery scheme executes redo(Ti) and sets the value of all the data items modified by Ti to the new value as found in the log
- This is done only for those transactions for which there is a <Ti Commit> record present in the log

Copyright © 2004, Infosys Technologies Ltd

If there is a failure, the recovery subsystem checks with the log to find out which transactions need to be redone. A transaction needs to be redone iff both the <Ti start> and <Ti commit> records are found in the log

Consider the following cases and the way recovery is handled in each case:

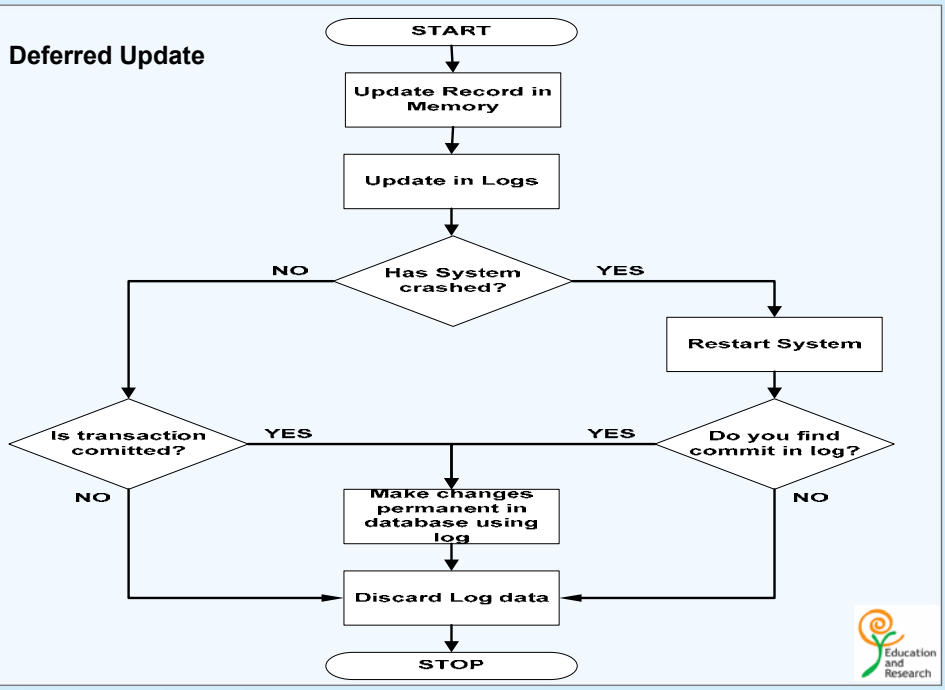
<T0 START>		<T0 START>		<T0 START>	
<T0, A, 900>		<T0, A, 900>		<T0, A, 900>	
<T0, B, 2100>	<T0, B, 2100>	<T0, B, 2100>			
.....C R A S H.....		**<**T0 Commit**>**	**<**T0 Commit**>**		
		<T1 Start>		<T1 Start>	
	(i)	<T1, C, 1100>	<T1, C, 1100>		
		..C R A S H.....	<T1 Commit>		
		(ii)		**.....C R A S H.....**	
				(iii)	

Case i: Since there are no <Ti start> , <Ti commit> pairs found, the log is just ignored.

Case ii: For T0, there is a <T0 start> <T0 Commit> pair of records found. So, redo(T0) is performed

Case iii: For both T0 and T1, the pair of records is found and hence redo(T0) and redo(T1) are performed so that the database is left at a consistent state.

Deferred Update



Deferred DB Modification

		Log Records	Database
T1	Read(A,a1)	< T1 Starts>	
	a1=a1-50		
	Write(A,a1)	< T1,A,950>	
	Read(B,b1)		
	b1=b1+50		
	Write(B,b1)		A=950
	Commit	< T1,B,2050>	
		< T1 Commit>	B=2050
T2	read(C,c1)	< T2 Starts>	
	c1=c1-100		
	Write(C,c1)	< T2,C,700>	C=700
	Commit	< T2 Commit>	

Occurrence of failure (Example)

<T0 Starts>
<T0, A,950>
<T0,B,2050>
CRASH

Recovery: No Action

<T0 Starts>
<T0, A,950>
<T0,B,2050>
<T0 Commits>
<T1 Starts>
<T1,C,600>
CRASH

Recovery: redo(T0)

<T0 Starts>
<T0, A,950>
<T0,B,2050>
<T0 Commits>
<T1 Starts>
<T1,C,600>
<T1 Commits>
CRASH

Recovery:Redo(T0),
Redo(T1)

Immediate update scheme (1 of 2)

- Allows the modifications to be output to the database while the transaction is in the ACTIVE state.
- This uses two procedures :**UNDO** and **REDO**

Copyright © 2004, Infosys Technologies Ltd

We require both the old value and the new value of a data item in this scheme.

Consider the same example that we used in the deferred update scheme:

A= 1000

B= 2000

C= 1000

Log	Database
<T0 start>	
<T0 A,1000,900>	
<T0 B, 2000,2100>	
	A= 900
	B= 2100
<To Commit>	
<T1 start>	
<T1 C, 1000,1100>	
	C= 1100
<T1 Commit>	

Note that the value of the data item is changed in the database immediately after the write operation is logged in the log . Ie even when the transaction is in the active state and has not entered the ‘partially committed’ state.

Immediate update scheme (2 of 2)

- The transaction is undone, if the log contains <Ti start> but does not contain <Ti COMMIT>
- The transaction need to be redone, if the log contains both the records <Ti START> and <Ti COMMIT>

Copyright © 2004, Infosys Technologies Ltd

The recovery scheme uses two procedures:
Undo(Ti): restores the value of all data items updated by Ti to old values
Redo(Ti): sets value of all data items to new values

If the start record is found but commit record is missing for a transaction, then undo should be performed on such transaction
If both start and commit are present, then redo should be done.

Now return to our previous example of the three cases of showing crash at different stages :

<T0 START>		<T0 START>	<T0 START>
<T0, A, 1000,900>	<T0, A, 1000,900>	<T0, A, 1000,900>	
<T0, B, 2000,2100>	<T0, B, 2000,2100>	<T0, B, 2000,2100>	
.....C R A S H.....	<T0 Commit>	<T0 Commit>	
		<T1 Start>	<T1 Start>
	(i)	<T1, C, 1000,1100>	<T1, C, 1000,1100>
		..C R A S H.....	<T1 Commit>
		(ii)	**.....C R A S H.....**
			(iii)

Case i: Since there are is only a <Ti start> but no <Ti commit>, transaction T0 has to be undone
Case ii: For T0, there is a <T0 start> <T0 Commit> pair of records found. So, redo(T0) is performed . But for T1, only start record is found but no commit record. So, undo has to be done.
Case iii: For both T0 and T1, the pair of records is found and hence redo(T0) and redo(T1) are performed so that the database is left at a consistent state.

The key difference between deferred updation and immediate updation is that, in deferred scheme, if the commit record is missing, the log is omitted but in the immediate scheme, since the chages are reflected on the database immediately, if the trnasaction crashes before committing, we have to undo the effect of the transaction and hence an undo operation is performed.

The next few slides illustrate these ideas with examples

Example

T1

Read(A,a1)
a1=a1-50
Write(A,a1)
Read(B,b1)
b1=b1+50
Write(B,b1)

A= 1000

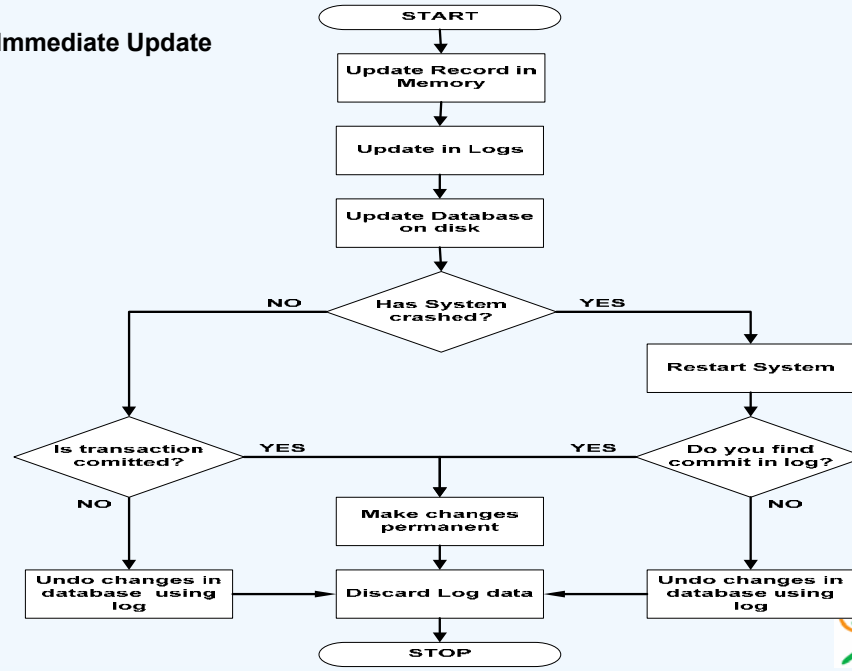
B = 2000

T2

read(C,c1)
c1=c1-100
Write(C,c1)

C=700

Immediate Update



Immediate DB Modification

		Log Records	Database
T1	Read(A,a1)	< T1 Starts>	
	a1=a1-50		
	Write(A,a1)	< T1,A,1000,950>	A=950
	Read(B,b1)		B=2050
	b1=b1+50		
	Write(B,b1)	< T1,B,2000,2050>	
	Commit	< T1 Commit>	
T2	read(C,c1)	< T2 Starts>	C=600
	c1=c1-100		
	Write(C,c1)	< T2,C,700,600>	
	Commit	< T2 Commit>	

Occurrence of failure (Example)

<T0 Starts>
<T0, A,1000,950>
<T0,B,2000,2050>
CRASH

Recovery: undo(T0)

<T0 Starts>
<T0, A,1000,950>
<T0,B,2000,2050>
<T0 Commits>
<T1 Starts>
<T1,C,700,600>
CRASH

Recovery:
undo(T1)
redo(T0)

<T0 Starts>
<T0, A,1000,950>
<T0,B,2000,2050>
<T0 Commits>
<T1 Starts>
<T1,C,700,600>
<T1 Commits>
CRASH

Recovery:
Redo(T0)
Redo(T1)

Checkpoints

- Problem
  - Searching process is time consuming
  - Most transactions might have to be redone
- The system regularly performs checkpoints
- The transactions are not allowed to perform any updates while the checkpoints are in progress.
- The presence of a checkpoint allows the system to streamline its recovery process

Copyright © 2004, Infosys Technologies Ltd

To reduce the time consumed in performing redo of all the transactions, system uses the concept of **Checkpoints**. Whether it uses deferred or immediate update, system performs checkpoint periodically. The following actions take place during checkpointing:

1. All log records residing on the primary memory are copied to stable storage
2. All modified buffer blocks are written to stable storage
3. A log record viz <checkpoint> is written onto the log stored on stable storage

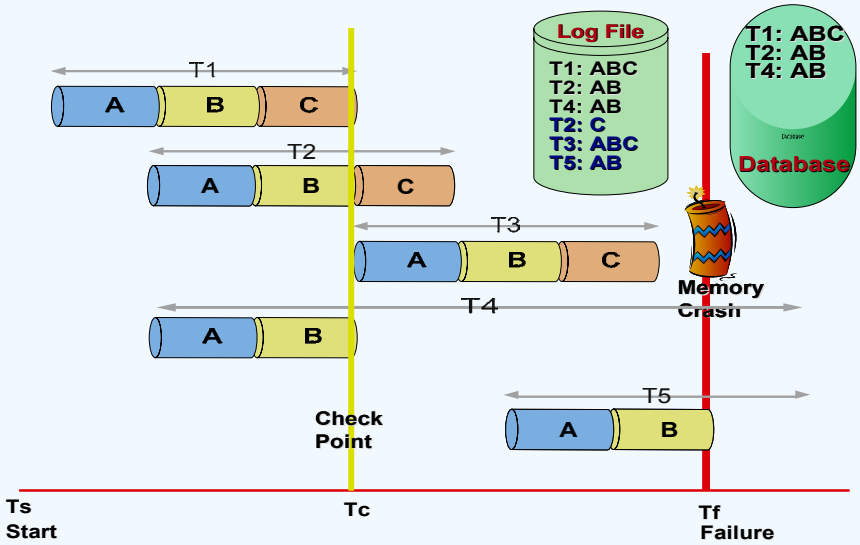
Now, when failure happens, the recovery process functions as below:

The log is searched from backwards

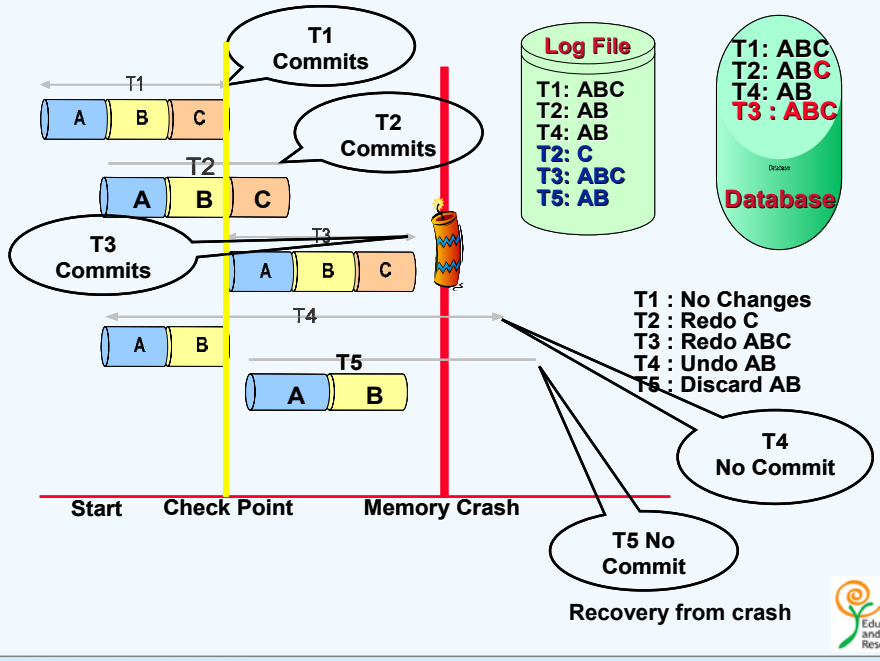
A hit is made on the <checkpoint > record that would be encountered during the searh (since we are searching from backwards, it would be the last <checkpoint> record in the log)

Now the transaction which has written its start record just before this checkpoint must be the most recently committed transaction. We need to redo only that transaction and any other transaction follows this, which has written its start record but no any commit record should be undone.

This process of undo and nullifying the effect of a failed transaction is called '**Rollback**'



Check Point Scenario



Recovery from crash

OLAP

- An organization's success also depends on its ability to analyze data and to make intelligent decisions that would potentially affect its future. Systems that facilitate such analysis are called On Line Analytical Processing (OLAP) systems.

Difference between OLTP and OLAP

	OLTP	OLAP
Definition	On Line Transaction Processing	On Line Analytical Processing
Data	Dynamic (day to day transaction / operational data)	Static (historical data)
Data Atomicity	Data is stored at microscopic level	Data is aggregated or summarized and stored at the higher level
Normalization	Normalized Databases to facilitate insertion, deletion and updation	De-normalized Databases to facilitate queries and analysis
History	Old data is purged or archived	Historical data stored to enable trend analysis and future predictions
Queries	Simple queries and updates Queries use small amounts of data (one record or a few records) Example: update account balance enroll for a course	Complex queries Queries use large amounts of data Example: Total annual sales for north region Total monthly sales for north region

Copyright © 2004, Infosys Technologies Ltd

Difference between OLTP and OLAP

Updates	Updates are frequent	Updates are infrequent
Response time	Fast response time is important Data must be up-to-date, consistent at all times	Transactions are slow Queries consume a lot of bandwidth
Joins in queries	Joins are more and complex as tables are normalized An OLTP system aims at one specific process Example: ordering from an online store	Joins are few and simple as tables are de-normalized An OLAP integrates data from different processes Example: Combines sales, inventory and purchasing data
Data models	Complex data models, many tables	Simple data models, fewer tables
Focus	OLTP focuses on performance	OLAP focuses on flexibility and broader scope

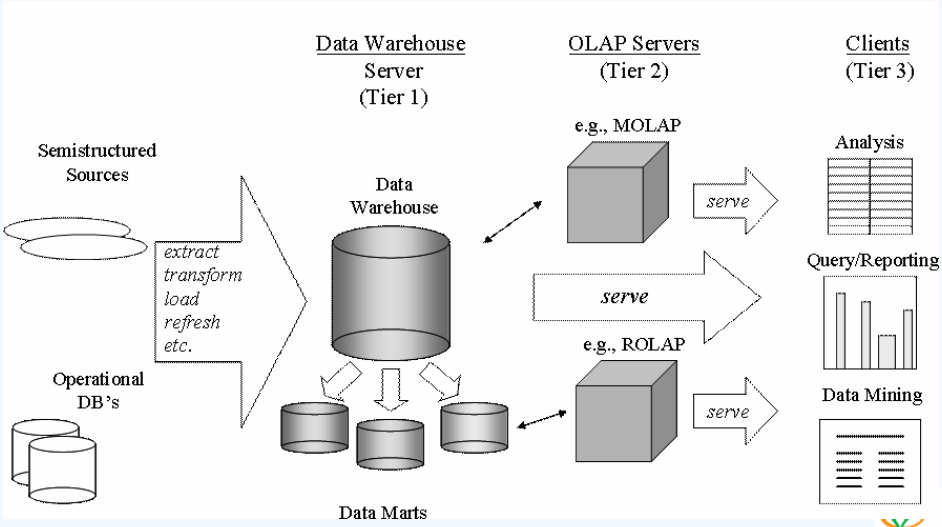
Data Warehouse

A data warehouse is a repository which stores integrated information for efficient querying and analysis. Data warehouse has data collected from multiple, disparate sources of an organization. It is the basis for decision support and data analysis systems.

Characteristics of Data Warehouse:

- **Subject-oriented:** means that all data pertinent to a subject/ business area are collected and stored as a single unit
- **Integrated:** means that data from multiple disparate sources are transformed and stored in a globally accepted fashion
- **Static/non-volatile:** means data once entered into the warehouse does not change frequently. It is periodically updated if required
- **Time variant:** Data warehouse maintains historical data which are used to analyse the business or market trends and facilitate future predictions

Data Warehouse Architecture



Data Warehousing Terminology

- **Data sources:** An organization has many functional units with their own data. Data from all such sources have to be consolidated and put into a consistent form that would reflect the business of an organization as a whole. These sources of data for a data warehouse are known as data sources or operational data sources.

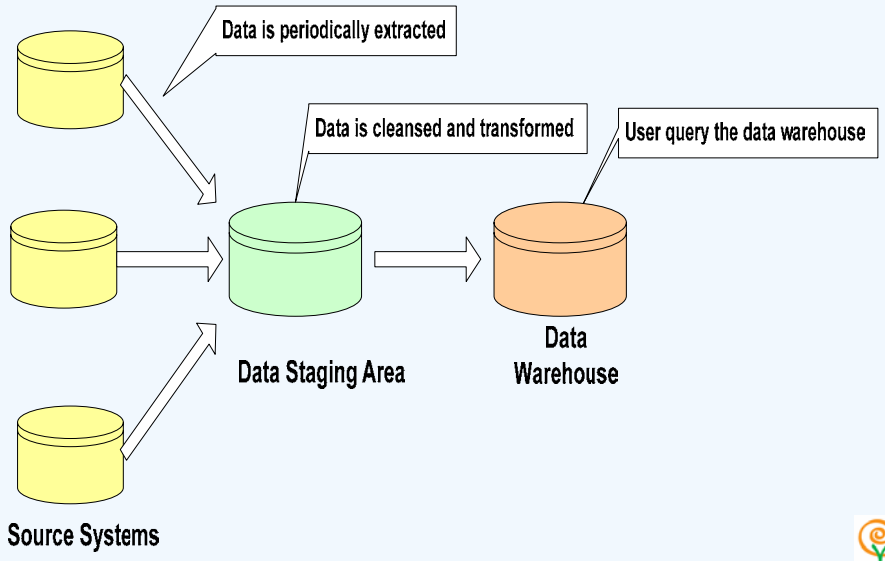
- **Metadata:** Metadata is the information about the data. This is the layer of the data warehouse which stores the information like the source data, transformed data, date and time of data extraction, target databases, data and time of data loading, etc.
- **Measure attributes:** A numerical value that can be summarized or can be aggregated upon.
- **Dimension attributes:** Dimensions can be defined as the perspectives used for looking at the data. Dimensions are the answer to the question "How do you want to see your data?"

Data Collection for Data Warehouse Applications

Extraction, transformation and loading (ETL): ETL is the most important step in Data Warehousing.

Definition of ETL: Extract, transform and load process is described as the process of selecting, migrating, transforming, cleansing and converting mapped data from the operational environment to data warehouse environment.

ETL Process

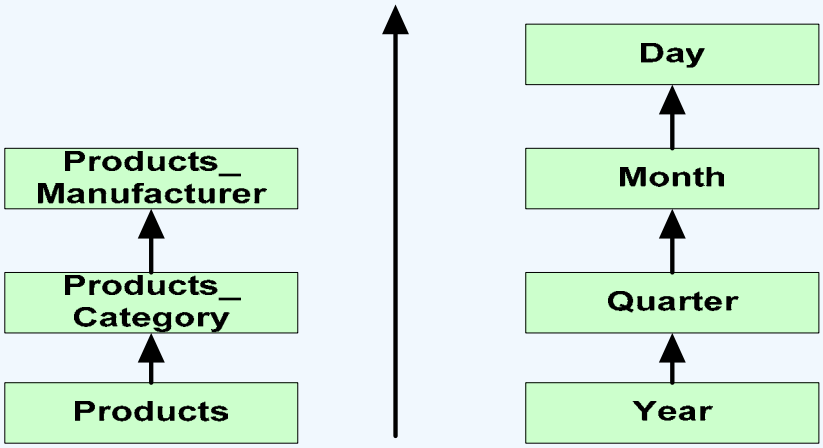

Data is periodically extracted
Data is cleansed and transformed
User query the data warehouse
Data Staging Area
Data Warehouse
Source Systems

Storing of data in Data warehouse

- **Dimensional Modeling:** The dimensional modeling is also known as star schema because in dimensional modeling there is a large central fact table with many dimension tables surrounding it.
- **Fact Tables:** Each data warehouse or data mart includes one or more fact tables. A fact table is central to a star or snowflake schema, and captures the data that measures the organization's business operations. Fact tables usually contain large numbers of rows.
- **Dimension Tables:** Dimension tables contain attributes that describe fact records in the fact table. Some of these attributes provide descriptive information; others are used to specify how fact table data should be summarized to provide useful information to the analyst. Dimension tables contain hierarchies of attributes that aid in summarization.
- **Cube:** The OLAP tools allows you to turn data stored in relational databases into meaningful, easy to navigate business information by creating data cube. The dimensions of a cube represent distinct categories for analyzing business data. Categories such as time, geography or product line breakdowns are typical cube dimensions.

Storing of data in Data warehouse

- **Dimension hierarchies:** The product dimension contains individual products. Products are classified into categories, and further classified according to manufacturer. The hierarchy for the dimension is stored in the dimension table.

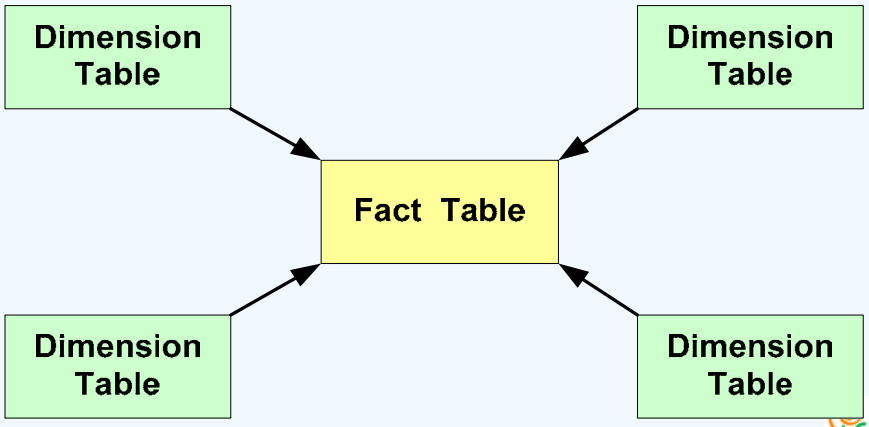


Schemas for dimensional modeling

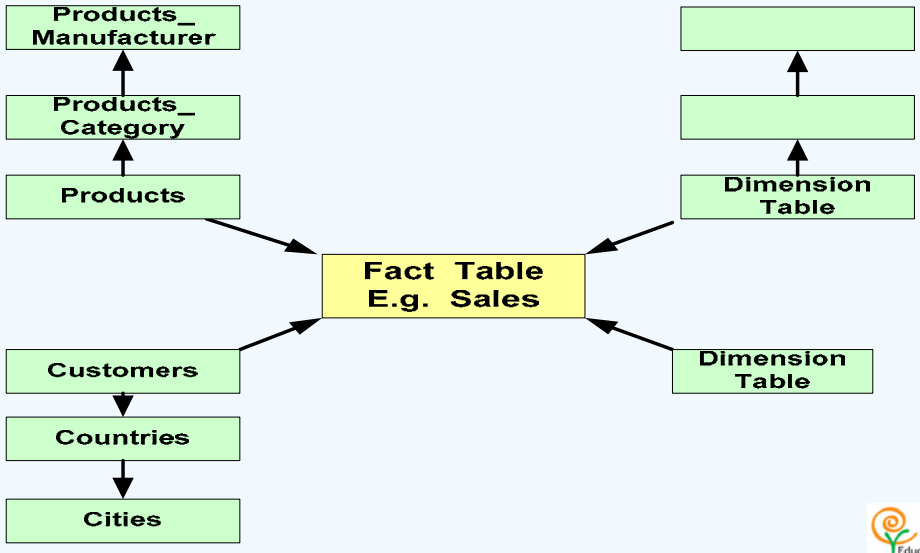
- **Star schema**: It is the simplest data warehouse schema. It resembles a star. The center of the star consists of one or more fact tables and the points radiating from the center are the dimension tables.

- **Snowflake Schema:** It is a complex data warehouse schema. The snowflake schema has a single, central fact table surrounded by normalized dimension hierarchies. Each dimension level is represented in a table.

Star Schema

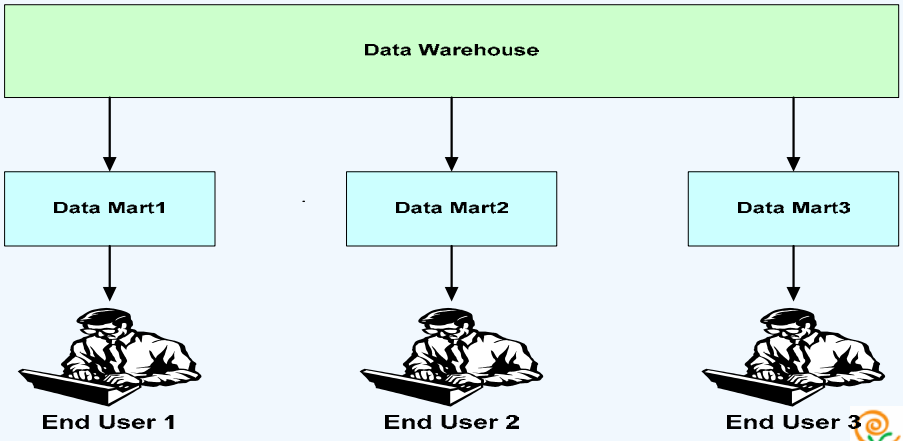

Star Schema
Dimension Table
Dimension Table
Fact Table
Dimension Table
Dimension Table

Snowflake Schema



Data Mart

A data mart is a focused subset of a data warehouse that deals with a single area of data and is organized for quick analysis. It can be a small data warehouse itself.



Advantages of Data Marts:

- It focuses on presentation rather than the organization of data
- It facilitates data reporting
- It provides meaningful reports to the users pertaining to their business area thereby allowing them to view and concentrate only on the data that is related to their business area
- It makes the data design simpler and easier. It breaks the whole design into several smaller sub units which is beneficial to the customers and the development team. It is also easier to maintain
- Reporting of data becomes faster and more efficient because reporting is generally done at the sub unit level and data marts assist in faster retrieval compared to querying the entire data warehouse
- It helps in incrementally building up the enterprise data warehouse
- It helps to ensure security

Difference between Data Warehouse & Data Mart

Data Warehouse	Data Mart
A data warehouse is a repository which stores integrated information from multiple disparate sources for efficient querying and analysis	A data mart is a focused subset of a data warehouse that deals with a single area of data and is organized for quick analysis.
It is concerned with the organization of data and offers little concern regarding the presentation of data to the customers	It is concerned mainly with the presentation of data to the customers rather than the organization of data in the data warehouse
There is usually a central data warehouse system	There can be several data marts that operate on the central data warehouse
Data Warehouse is used on an enterprise level	Data Mart is used on a business division / department level
Data Warehouse contains data from heterogeneous sources for analysis	Data Mart only contains the required subject specific data for local analysis

Popular tools available for data warehousing

Reporting / Analysis Tools:

- Micro Strategy: DSS Agent / Server
- Cognos: Improptu
- Brio Technology: Brio Query
- Seagate Software: Crystal Reports

ETL:

- Data Junction
- Microsoft DTS (Available with SQL Server 7.0 and above)
- Oracle Warehouse Builder
- Informatica: PowerMart
- IBM: DataPropagator
- Acta: ActaWorks
- **Databases:**

MDDB

  - Oracle Express
  - SAS

Summary

- Timestamping technique achieves serializability based on timestamps
- Timestamping avoids deadlock
- Database recovery is based on the concept of shadowing or logging
- Log based recovery scheme could be deffered or immediate
- An OLAP application requires historical data because an analysis is generally based on a substantial amount of historical data to enable trend analysis and future predictions
- A data warehouse is a repository which stores integrated information for efficient querying and analysis

Summary

- Extract, transform ad load process (ETL) is described as the process of selecting, migrating, transforming, cleansing and converting mapped data from the operational environment to data warehouse environment.
- A data mart is a focused subset of a data warehouse that deals with a single area of data and is organized for quick analysis.
- Star schema is the simplest data warehouse schema. It resembles a star. The center of the star consists of one or more fact tables and the points radiating from the center are the dimension tables.
- Snowflake schema has a single, central fact table surrounded by normalized dimension hierarchies.

Thank You!